

Table of Content

- [FreeRTOS Cheat Sheet for STM32](#)
- [FreeRTOS / CMSIS-RTOS2 Cheat Sheet for STM32](#)

FreeRTOS Cheat Sheet for STM32

Target: STM32 + STM32CubeMX + STM32CubeIDE **API:** Native FreeRTOS API (`xTaskCreate`, `xQueueSend`, `xSemaphoreTake`, ...) **Not covered:** CMSIS-RTOS V2 wrapper API

1. FreeRTOS — The Basic Idea

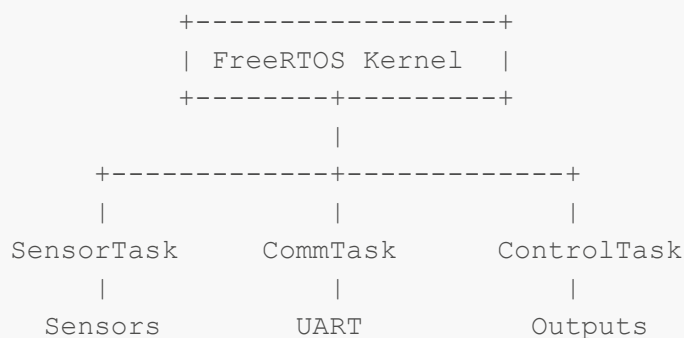
FreeRTOS is a **real-time operating system kernel** for embedded systems.

Without an RTOS, a typical embedded application might look like:

```
while (1)
{
    ReadSensors();
    ProcessData();
    HandleCommunication();
    CheckButtons();
    UpdateOutputs();
}
```

The application is responsible for deciding **when** each piece of code runs.

With FreeRTOS, you divide the application into independent **tasks**:



The kernel's scheduler decides which task gets CPU time.

The important idea is:

Tasks execute independently, but they need synchronization and communication mechanisms to work together safely.

The four mechanisms you will use constantly are:

Mechanism	Main purpose
Task	Execute application code independently
Queue	Transfer data between tasks
Semaphore	Signal an event/resource
Mutex	Protect a shared resource

2. Tasks

What is a task?

A task is essentially an independent execution context managed by FreeRTOS.

A task has:

- its own stack
- its own priority
- its own state
- its own program counter
- CPU registers saved/restored by the scheduler

A task function typically looks like this:

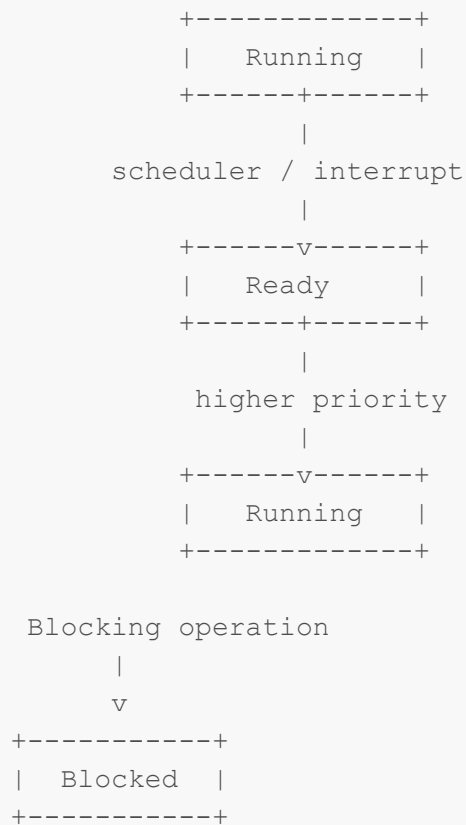
```
void MyTask(void *argument)
{
    while (1)
    {
        // Do something

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
```

The task normally contains an **infinite loop**.

Task states

A task can be in several states:



Typical states:

Running

Currently executing on the CPU.

Ready

Could execute, but another task currently has the CPU.

Blocked

Waiting for something:

```

vTaskDelay(...)
xQueueReceive(...)
xSemaphoreTake(...)

```

This is extremely important.

A well-designed RTOS application spends a lot of time with tasks **blocked**, rather than constantly polling.

3. Creating a Task

The native FreeRTOS API is:

```
xTaskCreate()
```

Example:

```
TaskHandle_t myTaskHandle;

xTaskCreate(
    MyTask,
    "MyTask",
    256,
    NULL,
    2,
    &myTaskHandle
);
```

Parameters:

```
xTaskCreate(
    task_function,
    task_name,
    stack_size,
    task_parameter,
    priority,
    task_handle
);
```

Example task:

```
void MyTask(void *argument)
{
    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

This creates a task which toggles an LED every 500 ms.

4. Task Priority

FreeRTOS uses priorities to determine which ready task executes.

For example:

```
Priority 3    CommunicationTask
Priority 2    ControlTask
Priority 1    LEDTask
Priority 0    IdleTask
```

If all three tasks are ready:

```
CommunicationTask
    ↓
    gets CPU
```

A higher-priority ready task normally preempts a lower-priority task.

Important rule

Don't simply give everything a high priority.

A sensible system might look like:

```
High priority
|
|— time-critical control
|— communication processing
|
|— normal application
|— diagnostic / logging
|
Low priority
```

5. `vTaskDelay()`

One of the most frequently used APIs:

```
vTaskDelay(pdMS_TO_TICKS(100));
```

This blocks the current task for 100 ms.

For example:

```
void LedTask(void *argument)
{
    while (1)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

The important point is that this **does not waste CPU time**.

While `LedTask` is blocked:

```
CPU
|
+-- LedTask
|   |
|   +--- BLOCKED
|
+-- OtherTask
|
+-- OtherTask
```

6. Periodic Tasks — `vTaskDelayUntil()`

For periodic tasks, `vTaskDelayUntil()` is often better.

Example:

```
void SensorTask(void *argument)
{
    TickType_t lastWakeTime = xTaskGetTickCount();

    while (1)
    {
        ReadSensor();

        vTaskDelayUntil(
            &lastWakeTime,
            pdMS_TO_TICKS(100)
        );
    }
}
```

This executes approximately every 100 ms.

Conceptually:

```

100ms      100ms      100ms
|-----|-----|-----|
Read       Read       Read

```

This avoids accumulated timing drift that can occur with:

```

ReadSensor();
vTaskDelay(100);

```

if `ReadSensor()` itself takes significant time.

7. Task Parameters

A task can receive a parameter.

Example:

```

void MyTask(void *argument)
{
    uint32_t value = *(uint32_t *)argument;

    while (1)
    {
        printf("Value = %lu\n", value);

        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

Creation:

```

uint32_t value = 123;

xTaskCreate(
    MyTask,
    "MyTask",
    256,
    &value,
    1,

```

```
        NULL
    );
```

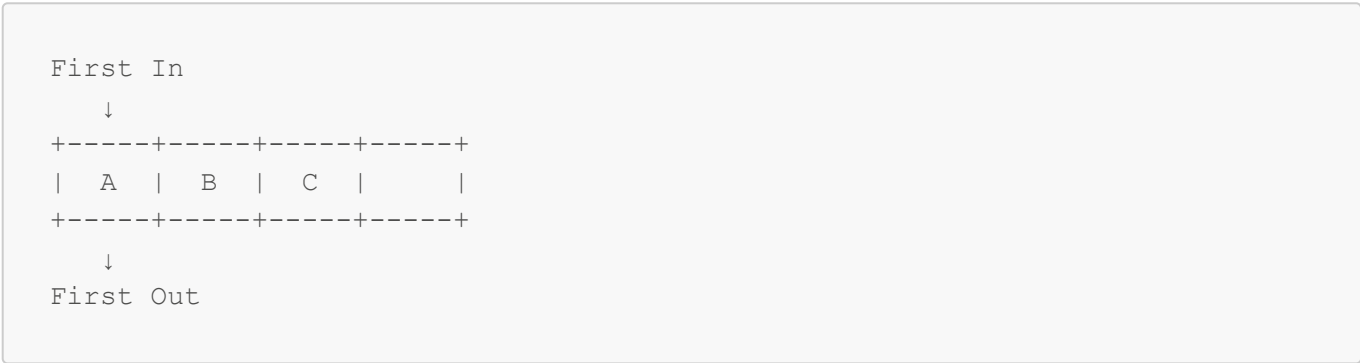
This is useful when creating several instances of the same task.

8. Queues

What is a queue?

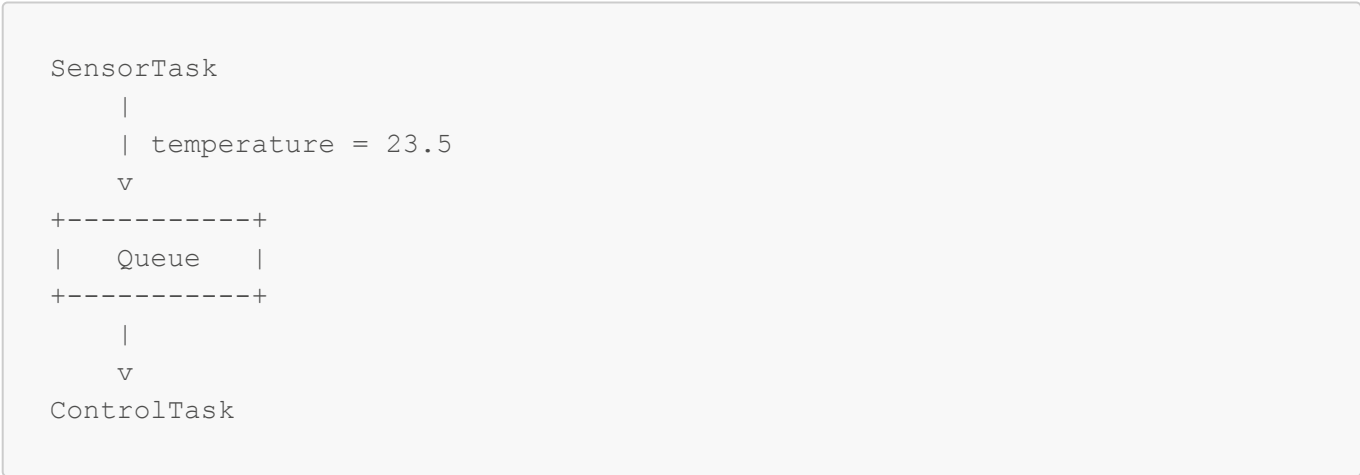
A queue is a **FIFO data structure managed by FreeRTOS**.

FIFO:



Queues are primarily used for **communication between tasks**.

Example:



This is one of the most useful FreeRTOS mechanisms.

9. Creating a Queue

Use:

```
xQueueCreate()
```

Example:

```
QueueHandle_t sensorQueue;

sensorQueue = xQueueCreate(10, sizeof(uint32_t));
```

This creates a queue containing:

```
10 elements
each element = 4 bytes
```

10. Sending to a Queue

```
uint32_t temperature = 235;

xQueueSend(
    sensorQueue,
    &temperature,
    portMAX_DELAY
);
```

The value is copied into the queue.

This is important:

FreeRTOS queues normally **copy the data**, they don't simply store your variable address.

11. Receiving from a Queue

Another task:

```
uint32_t temperature;

xQueueReceive(
    sensorQueue,
    &temperature,
    portMAX_DELAY
);
```

If no data is available, the task blocks.

This is exactly what you want.

```
ControlTask
  |
  v
xQueueReceive()
  |
  | no data
  v
BLOCKED
  |
  | queue receives data
  v
RUNNING
```

12. Complete Queue Example

Queue

```
QueueHandle_t sensorQueue;
```

Create it:

```
sensorQueue = xQueueCreate(
    10,
    sizeof(uint32_t)
);
```

Producer

```
void SensorTask(void *argument)
{
    uint32_t value;

    while (1)
    {
        value = ReadSensor();

        xQueueSend(
            sensorQueue,
            &value,
```

```

        portMAX_DELAY
    );

    vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

Consumer

```

void ControlTask(void *argument)
{
    uint32_t value;

    while (1)
    {
        if (xQueueReceive(
            sensorQueue,
            &value,
            portMAX_DELAY) == pdPASS)
        {
            ProcessSensorValue(value);
        }
    }
}

```

Conceptually:

```

SensorTask
|
| 123
| 124
| 125
v
+-----+
| Queue  |
| 123    |
| 124    |
| 125    |
+-----+
|
v
ControlTask

```

13. Queue Timeouts

The third parameter specifies how long the task should wait.

```
xQueueReceive(  
    queue,  
    &data,  
    pdMS_TO_TICKS(100)  
);
```

Meaning:

Wait up to 100 ms for data.

Other useful values:

0

Don't wait.

```
portMAX_DELAY
```

Wait indefinitely.

For most event-driven tasks:

```
portMAX_DELAY
```

is perfectly reasonable.

14. Queue Full

If the queue is full:

```
xQueueSend(...)
```

can block until space becomes available.

Example:

```
xQueueSend(  
    queue,  
    &data,
```

```
pdMS_TO_TICKS(50)
);
```

Wait up to 50 ms.

You can also explicitly check:

```
if (xQueueSend(queue, &data, 0) != pdPASS)
{
    // Queue full
}
```

15. Mutexes

What is a mutex?

A mutex is used to protect a **shared resource**.

For example, imagine two tasks use UART:

```
Task A ----+
           |
           v
          UART
           ^
           |
Task B ----+
```

Without protection:

```
Task A: "Hello"
Task B: "Temperature = 23"
```

You might end up transmitting:

```
HeTmeplelorature = 23
```

A mutex prevents this.

16. Creating a Mutex

```
SemaphoreHandle_t uartMutex;  
  
uartMutex = xSemaphoreCreateMutex();
```

Despite the name `SemaphoreHandle_t`, this handle can represent a mutex.

17. Taking a Mutex

Before using the resource:

```
xSemaphoreTake(  
    uartMutex,  
    portMAX_DELAY  
);
```

Then access the UART:

```
HAL_UART_Transmit(  
    &huart1,  
    data,  
    length,  
    100  
);
```

Finally:

```
xSemaphoreGive(uartMutex);
```

Complete example:

```
void TaskA(void *argument)  
{  
    while (1)  
    {  
        xSemaphoreTake(  
            uartMutex,  
            portMAX_DELAY  
        );  
  
        HAL_UART_Transmit(  

```

```
        &huart1,
        (uint8_t *) "Hello\r\n",
        7,
        100
    );

    xSemaphoreGive(uartMutex);

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}
```

Another task can use the same mutex:

```
void TaskB(void *argument)
{
    while (1)
    {
        xSemaphoreTake(
            uartMutex,
            portMAX_DELAY
        );

        HAL_UART_Transmit(
            &huart1,
            (uint8_t *) "Temperature\r\n",
            13,
            100
        );

        xSemaphoreGive(uartMutex);

        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

Only one task can own the mutex at a time.

18. Mutex vs Queue

This distinction is fundamental.

Queue

Used to **transfer data**.

```

Task A
  |
  | data
  v
Queue
  |
  | data
  v
Task B

```

Mutex

Used to **protect a resource**.

```

Task A ----+
          |
          v
        Mutex ----> UART
          ^
          |
Task B ----+

```

Think:

Queue = communication

Mutex = ownership/protection

19. Priority Inheritance

Mutexes have an important feature called **priority inheritance**.

Consider:

```

High priority Task
  |
  | wants UART
  v
  Mutex
  ^
  |
Low priority Task
owns it

```

Without priority inheritance, the high-priority task could be blocked by the low-priority task while a medium-priority task keeps running.

This is called **priority inversion**.

FreeRTOS mutexes implement priority inheritance.

This is one reason why you should use a **mutex**, rather than a binary semaphore, when protecting a shared resource.

20. Semaphores

A semaphore is primarily a **synchronization mechanism**.

There are two common types:

- Binary semaphore
- Counting semaphore

21. Binary Semaphore

A binary semaphore has two states:

```
EMPTY
|
| Give
v
AVAILABLE
|
| Take
v
EMPTY
```

It is useful for signaling:

```
Interrupt
|
| event occurred
v
Semaphore
|
v
Task
```

22. Binary Semaphore Example

Create:

```
SemaphoreHandle_t adcSemaphore;  
  
adcSemaphore = xSemaphoreCreateBinary();
```

Task:

```
void AdcTask(void *argument)  
{  
    while (1)  
    {  
        xSemaphoreTake(  
            adcSemaphore,  
            portMAX_DELAY  
        );  
  
        ProcessADC();  
    }  
}
```

Some other code signals the task:

```
xSemaphoreGive(adcSemaphore);
```

The task wakes up.

23. ISR → Task Synchronization

This is a particularly common STM32 pattern.

For example:

```
UART interrupt  
    |  
    | event  
    v  
Binary Semaphore  
    |  
    v  
UART Task
```

From an ISR you must use the `FromISR` API.

Example:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    if (GPIO_Pin == GPIO_PIN_0)
    {
        xSemaphoreGiveFromISR(
            buttonSemaphore,
            &xHigherPriorityTaskWoken
        );

        portYIELD_FROM_ISR(
            xHigherPriorityTaskWoken
        );
    }
}
```

The task:

```
void ButtonTask(void *argument)
{
    while (1)
    {
        xSemaphoreTake(
            buttonSemaphore,
            portMAX_DELAY
        );

        ButtonPressed();
    }
}
```

This is a very clean ISR/task architecture.

The ISR does minimal work:

```
ISR:
    detect event
    signal task
    exit
```

```
Task:
    perform actual processing
```

24. FromISR Functions

A very important rule:

Never call normal blocking FreeRTOS APIs from an ISR.

Instead use the corresponding `FromISR` function.

Examples:

Task context	ISR context
<code>xQueueSend()</code>	<code>xQueueSendFromISR()</code>
<code>xQueueReceive()</code>	Usually not used
<code>xSemaphoreGive()</code>	<code>xSemaphoreGiveFromISR()</code>
<code>xTaskNotify()</code>	<code>xTaskNotifyFromISR()</code>

The reason is that an ISR cannot block.

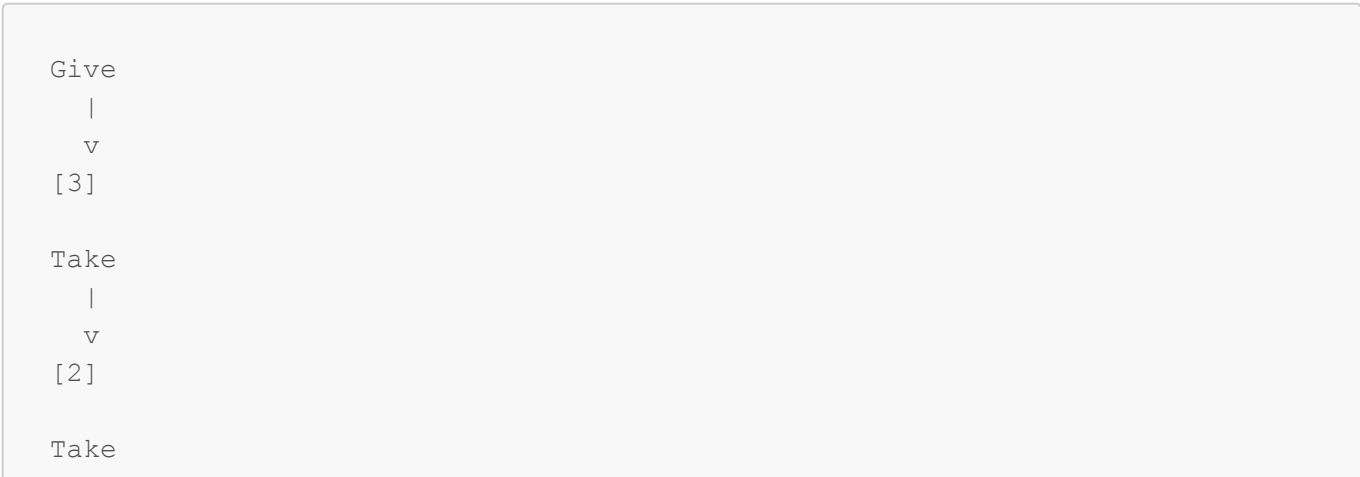
25. Counting Semaphores

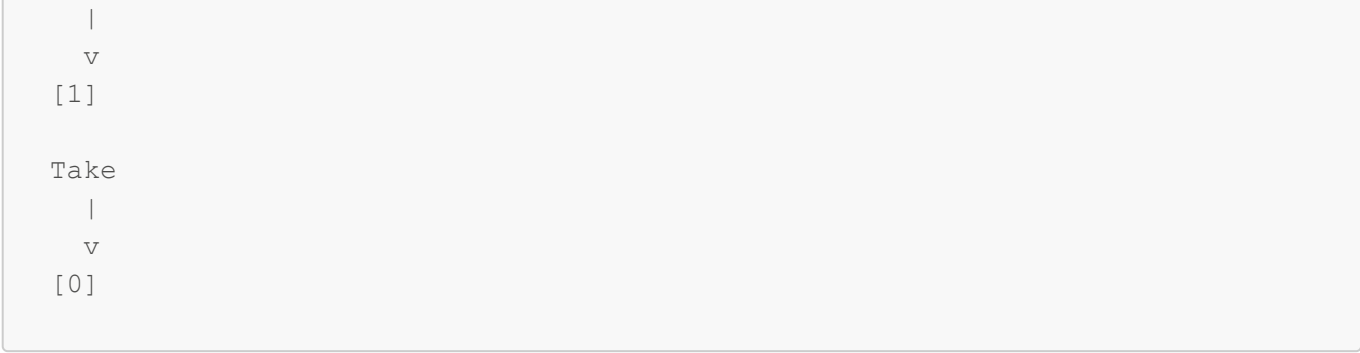
A counting semaphore can represent a number of available resources/events.

Example:

```
Semaphore count = 3
```

Three tasks/events can consume it:





Create:

```
SemaphoreHandle_t semaphore;

semaphore = xSemaphoreCreateCounting(
    10,
    0
);
```

Parameters:

```
maximum count = 10
initial count = 0
```

Useful examples include:

- counting events
- resource pools
- multiple identical hardware resources

26. Mutex vs Binary Semaphore

They may look similar:

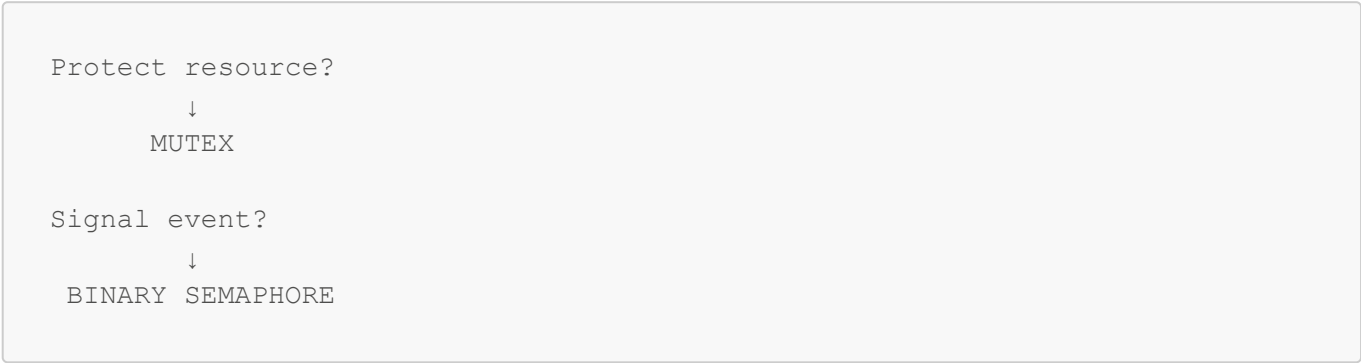
```
xSemaphoreTake()
xSemaphoreGive()
```

But they have different semantics.

	Mutex	Binary Semaphore
Purpose	Protect resource	Synchronize/event
Ownership	Yes	No

	Mutex	Binary Semaphore
Priority inheritance	Yes	No
ISR give	No	Yes
Typical use	UART, SPI, shared data	ISR → task event

Rule of thumb:



27. Task Notifications

Although not requested explicitly, this is worth knowing.

FreeRTOS task notifications are often an even lighter-weight alternative to binary semaphores.

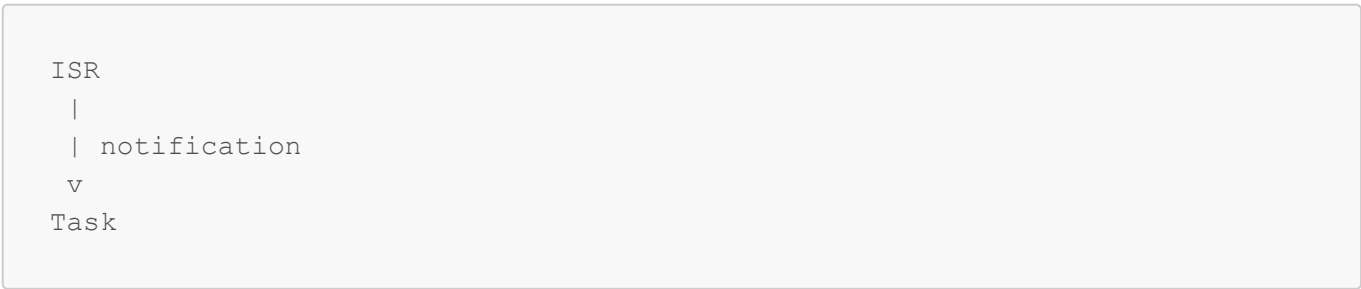
Example:

```
xTaskNotifyGive(taskHandle);
```

Task:

```
ulTaskNotifyTake(
    pdTRUE,
    portMAX_DELAY
);
```

Conceptually:



Task notifications are extremely efficient because the notification value is stored directly inside the task's control block.

For many simple ISR → task signals:

```
Task Notification
```

is preferable to creating a separate semaphore.

28. Queues vs Notifications vs Semaphores

A useful mental model:

```
Need to transfer DATA?
    |
    v
  QUEUE

Need to signal EVENT?
    |
    v
SEMAPHORE / NOTIFICATION

Need to protect RESOURCE?
    |
    v
  MUTEX
```

29. Critical Sections

FreeRTOS also provides critical sections.

```
taskENTER_CRITICAL();

/* critical code */

taskEXIT_CRITICAL();
```

This temporarily prevents certain interrupts/context switches.

Use this for **very short operations**.

Example:

```
taskENTER_CRITICAL();

sharedCounter++;

taskEXIT_CRITICAL();
```

Do not put lengthy operations inside:

```
taskENTER_CRITICAL();

/* DON'T */
HAL_Delay(100);
HAL_UART_Transmit(...);
complexCalculation();

taskEXIT_CRITICAL();
```

Critical sections should generally be extremely short.

30. Delays vs Synchronization

Avoid this:

```
while (dataReady == 0)
{
    vTaskDelay(pdMS_TO_TICKS(10));
}
```

This is polling.

Better:

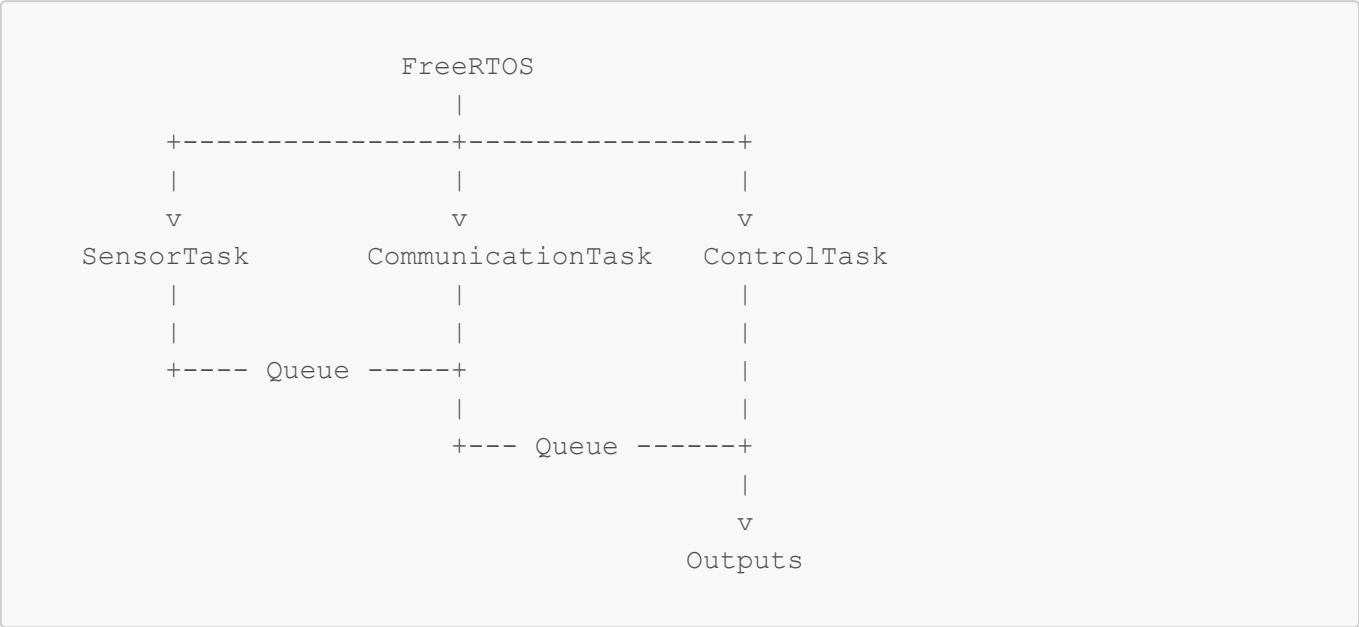
```
xSemaphoreTake(
    dataReadySemaphore,
    portMAX_DELAY
);
```

Now the task sleeps until the event actually happens.

This is one of the major advantages of an RTOS.

31. Typical STM32 Architecture

A real STM32 application might look like:



Hardware interrupts sit underneath:



32. STM32CubeMX Integration

When enabling FreeRTOS in CubeMX, CubeMX will generate the basic RTOS infrastructure.

You will typically see something like:



```
MX_FREERTOS_Init();
```

and generated task code.

CubeMX may generate CMSIS-RTOS V2 wrappers such as:

```
osThreadNew(...)
osDelay(...)
osMessageQueuePut(...)
```

If you want to use the **native FreeRTOS API**, include the FreeRTOS headers directly:

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "semphr.h"
```

Then use:

```
xTaskCreate(...)
xQueueCreate(...)
xQueueSend(...)
xSemaphoreTake(...)
vTaskDelay(...)
```

rather than:

```
osThreadNew(...)
osMessageQueuePut(...)
osSemaphoreAcquire(...)
osDelay(...)
```

33. Recommended Project Structure

For a larger STM32 project, avoid putting everything into `main.c`.

For example:

```
Core/
├─ Inc/
```

```
|   ├── app_tasks.h  
|   ├── sensor_task.h  
|   ├── communication_task.h  
|   └── control_task.h  
└── Src/  
    ├── app_tasks.c  
    ├── sensor_task.c  
    ├── communication_task.c  
    └── control_task.c
```

A task module might contain:

```
#include "FreeRTOS.h"  
#include "task.h"  
#include "queue.h"  
  
static TaskHandle_t sensorTaskHandle;  
  
void SensorTask(void *argument)  
{  
    while (1)  
    {  
        // Sensor processing  
  
        vTaskDelay(pdMS_TO_TICKS(100));  
    }  
}  
  
void SensorTask_Init(void)  
{  
    xTaskCreate(  
        SensorTask,  
        "Sensor",  
        256,  
        NULL,  
        2,  
        &sensorTaskHandle  
    );  
}
```

34. A Small Complete Example

Imagine an STM32 application with:

- ADC measurement
- processing task
- UART output

Architecture:

```
ADC
|
| measurement
v
SensorTask
|
| Queue
v
ProcessingTask
|
| Queue
v
CommunicationTask
|
v
UART
```

Queue definitions:

```
QueueHandle_t adcQueue;
QueueHandle_t resultQueue;
```

Initialization:

```
void App_Init(void)
{
    adcQueue = xQueueCreate(
        10,
        sizeof(uint16_t)
    );

    resultQueue = xQueueCreate(
        10,
        sizeof(uint32_t)
    );

    xTaskCreate(
        SensorTask,
        "Sensor",
        256,
        NULL,
        2,
        NULL
    );

    xTaskCreate(
```

```

        ProcessingTask,
        "Process",
        256,
        NULL,
        2,
        NULL
    );

    xTaskCreate(
        CommunicationTask,
        "Comm",
        256,
        NULL,
        1,
        NULL
    );
}

```

Sensor:

```

void SensorTask(void *argument)
{
    uint16_t adcValue;

    while (1)
    {
        adcValue = ReadADC();

        xQueueSend(
            adcQueue,
            &adcValue,
            portMAX_DELAY
        );

        vTaskDelayUntil(
            &(TickType_t){ xTaskGetTickCount() },
            pdMS_TO_TICKS(100)
        );
    }
}

```

Processing:

```

void ProcessingTask(void *argument)
{
    uint16_t adcValue;
    uint32_t result;

    while (1)

```

```
{
    if (xQueueReceive(
        adcQueue,
        &adcValue,
        portMAX_DELAY) == pdPASS)
    {
        result = ProcessADC(adcValue);

        xQueueSend(
            resultQueue,
            &result,
            portMAX_DELAY
        );
    }
}
```

Communication:

```
void CommunicationTask(void *argument)
{
    uint32_t result;

    while (1)
    {
        if (xQueueReceive(
            resultQueue,
            &result,
            portMAX_DELAY) == pdPASS)
        {
            SendResultUART(result);
        }
    }
}
```

This gives a clean **producer** → **consumer** architecture.

35. Common FreeRTOS API Cheat Sheet

Tasks

```
xTaskCreate()
vTaskDelete()
vTaskDelay()
vTaskDelayUntil()
vTaskSuspend()
```

```
vTaskResume ()  
xTaskGetTickCount ()
```

Queues

```
xQueueCreate ()  
  
xQueueSend ()  
xQueueSendToBack ()  
xQueueSendToFront ()  
  
xQueueReceive ()  
  
xQueuePeek ()  
  
uxQueueMessagesWaiting ()  
uxQueueSpacesAvailable ()
```

ISR variants:

```
xQueueSendFromISR ()  
xQueueReceiveFromISR ()
```

Mutex / Semaphores

```
xSemaphoreCreateMutex ()  
  
xSemaphoreCreateBinary ()  
  
xSemaphoreCreateCounting ()  
  
xSemaphoreTake ()  
xSemaphoreGive ()
```

ISR:

```
xSemaphoreGiveFromISR ()  
xSemaphoreTakeFromISR ()
```

Task Notifications

```
xTaskNotify()  
xTaskNotifyGive()  
  
ulTaskNotifyTake()  
xTaskNotifyWait()  
  
xTaskNotifyFromISR()  
vTaskNotifyGiveFromISR()  
ulTaskNotifyTakeFromISR()
```

36. The Most Important Design Rules

1. Don't use delays for synchronization

Bad:

```
vTaskDelay(pdMS_TO_TICKS(100));  
if (dataReady)  
{  
    Process();  
}
```

Better:

```
xSemaphoreTake(dataReadySemaphore, portMAX_DELAY);  
Process();
```

2. Don't poll unnecessarily

Bad:

```
while (!rxComplete)  
{  
    vTaskDelay(1);  
}
```

Better:

```
xTaskNotifyWait(...);
```

or:

```
xSemaphoreTake(rxSemaphore, portMAX_DELAY);
```

3. Keep ISRs short

Prefer:

```
ISR
|
+-- capture information
+-- signal task
|
+-- return
```

rather than doing complex processing inside the ISR.

4. Don't hold mutexes longer than necessary

Bad:

```
xSemaphoreTake(uartMutex, portMAX_DELAY);

DoComplexCalculation();

HAL_Delay(100);

HAL_UART_Transmit(...);

xSemaphoreGive(uartMutex);
```

Better:

```
DoComplexCalculation();

xSemaphoreTake(uartMutex, portMAX_DELAY);

HAL_UART_Transmit(...);

xSemaphoreGive(uartMutex);
```

5. Be careful with stack size

Every task has its own stack.

```
xTaskCreate(  
    MyTask,  
    "Task",  
    256,  
    NULL,  
    2,  
    NULL  
);
```

The stack requirement depends heavily on what the task calls.

Large local arrays, printf, floating-point operations, etc. can significantly increase stack usage.

FreeRTOS provides mechanisms for checking stack usage, e.g.:

```
uxTaskGetStackHighWaterMark()
```

6. Avoid blocking the highest-priority task unnecessarily

If a high-priority task spends most of its time doing:

```
while (1)  
{  
    // huge processing loop  
}
```

then lower-priority tasks may get starved.

A task should generally either:

```
vTaskDelay(...)
```

or:

```
xQueueReceive(...);  
xSemaphoreTake(...);  
ulTaskNotifyTake(...);
```

when it has nothing to do.

37. A Practical Decision Tree

When writing a new piece of functionality, ask:

```
Does it need its own execution context?  
  |  
  YES  
  |  
  TASK
```

Then:

```
Does Task A need to send DATA to Task B?  
  |  
  YES  
  |  
  QUEUE
```

```
Does one task need to wait for an EVENT?  
  |  
  YES  
  |  
  NOTIFICATION / SEMAPHORE
```

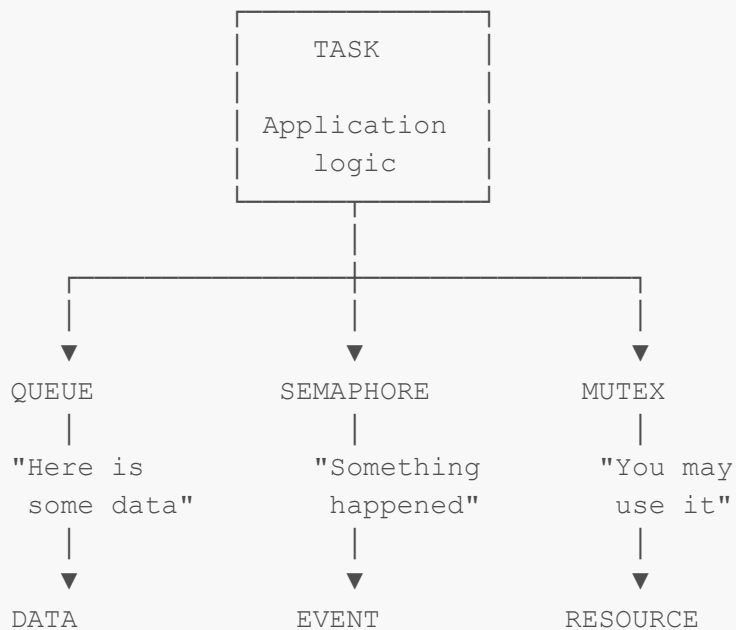
```
Does multiple code need access to the  
same RESOURCE?  
  |  
  YES  
  |  
  MUTEX
```

And:

```
Did the event originate from an ISR?  
  |  
  YES  
  |  
  Use FromISR API
```

38. One Mental Model to Remember

If you remember only this:



That covers a surprisingly large percentage of real-world FreeRTOS applications.

39. Recommended First STM32 FreeRTOS Experiment

For getting familiar with the kernel, I would build a tiny application with exactly **three tasks**:

```
LED Task
|
+-- toggles LED every 500 ms

Producer Task
|
+-- generates counter
|
+-- sends counter to queue

Consumer Task
|
+-- waits for queue
|
+-- prints received counter over UART
```

Then extend it:

```

Button ISR
|
+-- Binary semaphore
    |
    v
    ButtonTask
    |
    +-- changes LED mode

```

Finally add:

```

UART
^
|
Mutex
^
|
Task A + Task B

```

That small project will give you hands-on experience with the four fundamental FreeRTOS mechanisms:

```

Tasks
Queues
Semaphores
Mutexes

```

and, importantly, how they interact rather than just how their APIs look.

FreeRTOS / CMSIS-RTOS2 Cheat Sheet for STM32

Target: STM32 + STM32CubeMX + STM32CubeIDE **API:** CMSIS-RTOS2 (`osThreadNew`, `osMessageQueuePut`, `osMutexAcquire`, ...) **Underlying kernel:** FreeRTOS **Header:** `cmsis_os2.h`

1. FreeRTOS / CMSIS-RTOS2 — The Basic Idea

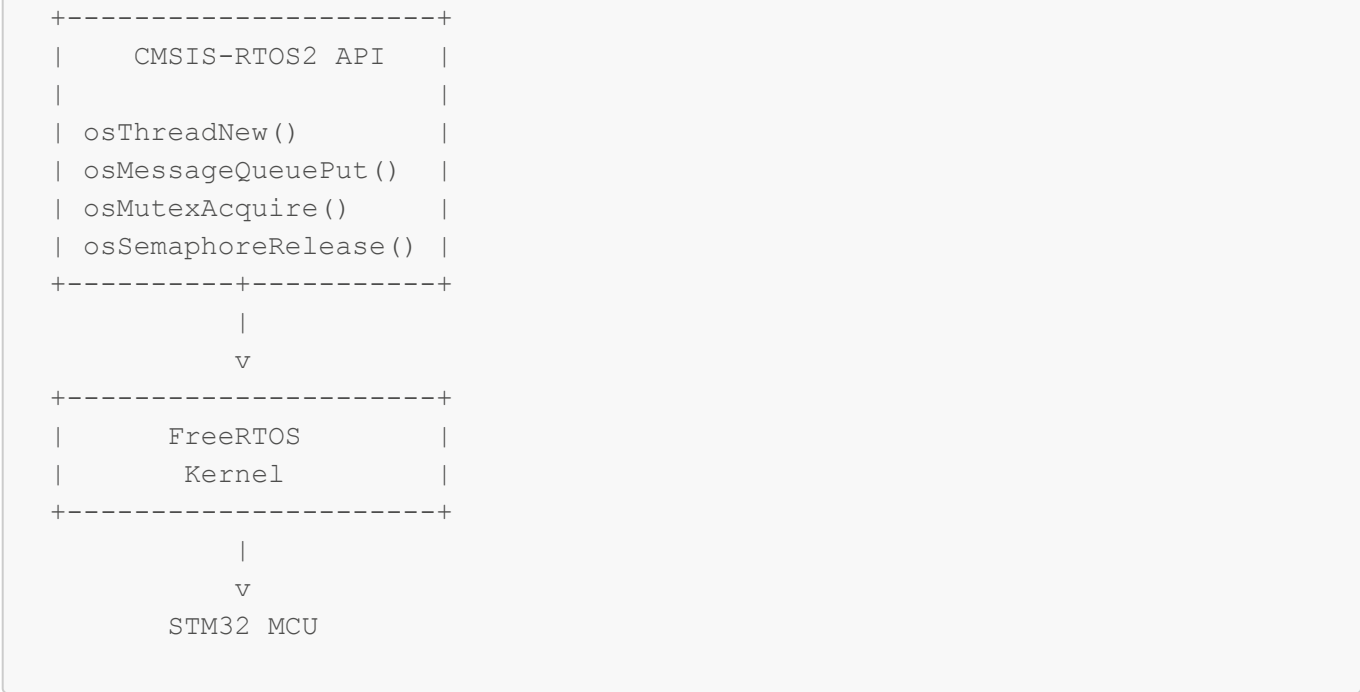
CMSIS-RTOS2 provides a **standardized API for RTOS functionality**.

In an STM32Cube project, the architecture can look like this:

```

Application
|
v

```



The important point is:

You program against the CMSIS-RTOS2 API instead of directly against the FreeRTOS API.

For example:

```
osThreadNew(...)
```

instead of:

```
xTaskCreate(...)
```

And:

```
osMessageQueuePut(...)
```

instead of:

```
xQueueSend(...)
```

The underlying concepts remain the same:

Mechanism	Main purpose
Thread	Execute application code independently

Mechanism	Main purpose
Message Queue	Transfer data between threads
Semaphore	Signal an event/resource
Mutex	Protect a shared resource

CMSIS-RTOS2 calls FreeRTOS "tasks" **threads**.

2. Threads

What is a thread?

A CMSIS-RTOS2 thread is essentially the same concept as a FreeRTOS task.

It provides an independent execution context with:

- its own stack
- its own priority
- its own state
- its own execution point
- CPU context managed by the RTOS

A thread function looks like:

```
void MyThread(void *argument)
{
    while (1)
    {
        // Do something

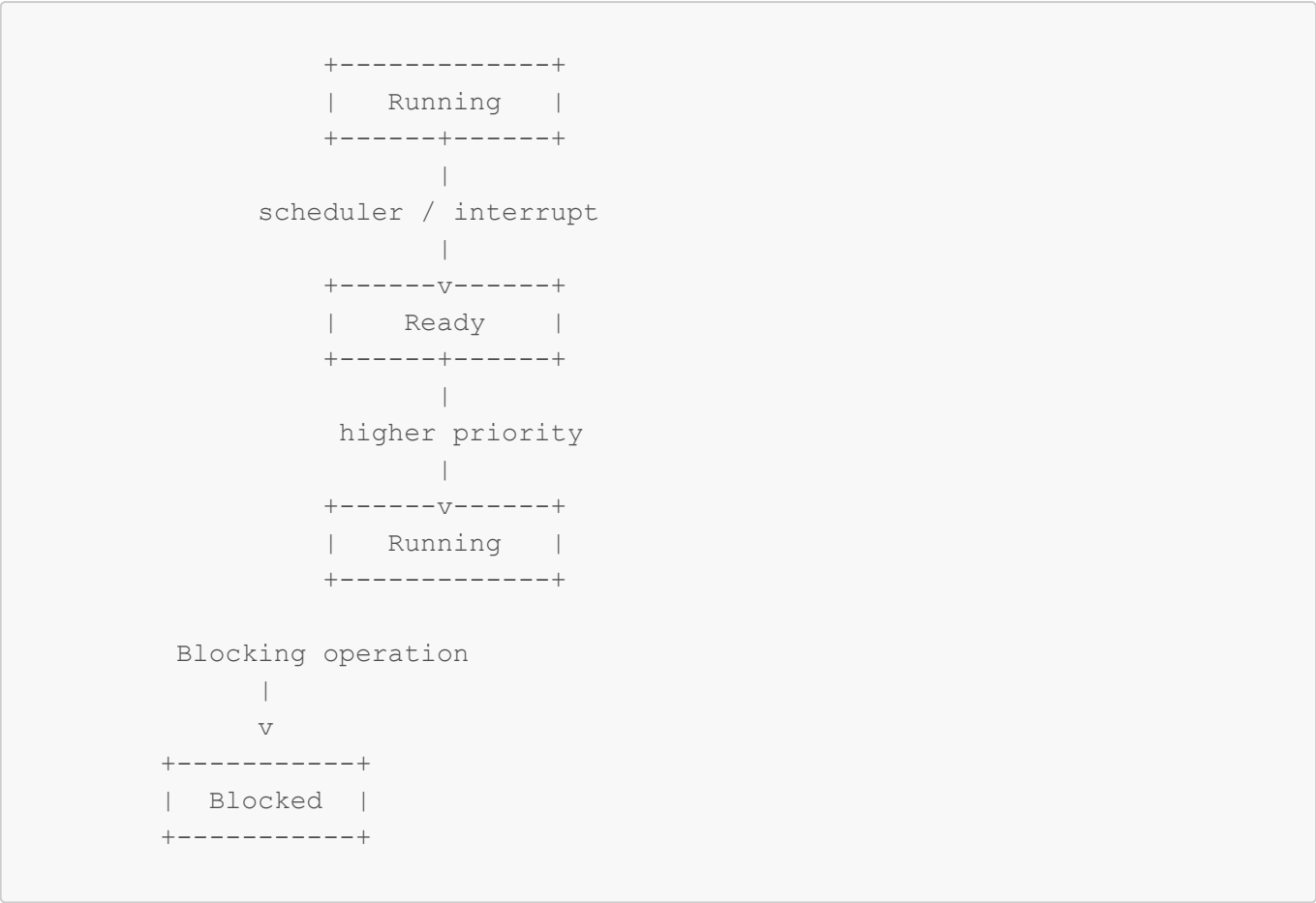
        osDelay(100);
    }
}
```

Notice the terminology:

FreeRTOS		CMSIS-RTOS2
Task	--->	Thread
TaskHandle	--->	osThreadId_t
vTaskDelay	--->	osDelay

Thread states

CMSIS-RTOS2 defines states such as:



The CMSIS-RTOS2 API explicitly defines states including:

```
osThreadReady
osThreadRunning
osThreadBlocked
osThreadTerminated
```

A thread becomes blocked when it waits for something such as a delay, semaphore, mutex or message queue.

3. Creating a Thread

The CMSIS-RTOS2 API uses:

```
osThreadNew()
```

Example:

```
osThreadId_t myThreadHandle;

myThreadHandle = osThreadNew(
    MyThread,
    NULL,
    NULL
);
```

Parameters:

```
osThreadNew(
    thread_function,
    argument,
    attributes
);
```

This is considerably simpler than:

```
xTaskCreate(
    MyTask,
    "MyTask",
    256,
    NULL,
    2,
    &myTaskHandle
);
```

In CMSIS-RTOS2, configuration such as the **name**, **priority** and **stack size** is normally provided through an attribute structure.

Example:

```
const osThreadAttr_t myThread_attributes = {
    .name = "MyThread",
    .stack_size = 512 * 4,
    .priority = (osPriority_t) osPriorityNormal
};

osThreadId_t myThreadHandle;

myThreadHandle = osThreadNew(
    MyThread,
    NULL,
    &myThread_attributes
);
```

The exact generated code in STM32CubeMX may look slightly different depending on the CubeMX version and configuration.

4. Thread Priority

CMSIS-RTOS2 uses `osPriority_t`.

For example:

```
.priority = osPriorityNormal
```

Some commonly used priorities are:

```
osPriorityIdle  
osPriorityLow  
osPriorityBelowNormal  
osPriorityNormal  
osPriorityAboveNormal  
osPriorityHigh  
osPriorityRealtime
```

Example:

```
const osThreadAttr_t communicationTask_attributes = {  
    .name = "Communication",  
    .stack_size = 512 * 4,  
    .priority = (osPriority_t) osPriorityHigh  
};
```

Compared with native FreeRTOS:

```
.priority = osPriorityHigh
```

instead of:

```
priority = 3
```

This makes application code more readable.

The CMSIS-RTOS2 specification reserves the lowest and highest special priorities for RTOS purposes, so application priorities should be selected from the normal user range.

5. `osDelay()`

The CMSIS equivalent of:

```
vTaskDelay()
```

is:

```
osDelay()
```

Example:

```
void LedThread(void *argument)
{
    while (1)
    {
        HAL_GPIO_TogglePin(
            GPIOA,
            GPIO_PIN_5
        );

        osDelay(500);
    }
}
```

This blocks the current thread for 500 ms.

It does **not** busy-wait.

While the LED thread is blocked:

```
CPU
|
+-- LedThread
|   |
|   +--- BLOCKED
|
+-- OtherThread
|
+-- OtherThread
```

6. Periodic Threads — `osDelayUntil()`

CMSIS-RTOS2 also provides:

```
osDelayUntil()
```

This is useful for periodic execution.

A typical pattern is:

```
void SensorThread(void *argument)
{
    uint32_t nextWakeTime = osKernelGetTickCount();

    while (1)
    {
        ReadSensor();

        nextWakeTime += 100;

        osDelayUntil(nextWakeTime);
    }
}
```

Conceptually:

```
100ms      100ms      100ms
|-----|-----|-----|
Read       Read       Read
```

This is preferable to simply doing:

```
ReadSensor();
osDelay(100);
```

for a periodic task because the execution time of `ReadSensor()` would otherwise add to the period.

`osDelayUntil()` uses the kernel tick count as its time base. CMSIS-RTOS2 provides `osKernelGetTickCount()` for retrieving that count.

7. Thread Parameters

A thread can receive an arbitrary pointer as its argument.

Example:

```
void MyThread(void *argument)
{
    uint32_t value = *(uint32_t *)argument;

    while (1)
    {
        printf(
            "Value = %lu\r\n",
            value
        );

        osDelay(1000);
    }
}
```

Creation:

```
uint32_t value = 123;

osThreadNew(
    MyThread,
    &value,
    NULL
);
```

This is equivalent to the parameter mechanism of:

```
xTaskCreate()
```

in native FreeRTOS.

The CMSIS-RTOS2 specification explicitly defines the `argument` parameter as arbitrary user data passed to the thread function.

8. Message Queues

CMSIS-RTOS2 calls them **Message Queues**.

They serve essentially the same purpose as FreeRTOS queues:

Transfer data safely from one thread to another.

Conceptually:

```

SensorThread
  |
  | temperature
  v
+-----+
| Message Queue |
+-----+
  |
  v
ControlThread

```

The CMSIS API is:

```

osMessageQueueNew()
osMessageQueuePut()
osMessageQueueGet()

```

Unlike the native FreeRTOS API, CMSIS-RTOS2 uses the word **message** consistently.

9. Creating a Message Queue

Use:

```
osMessageQueueNew()
```

Example:

```

osMessageQueueId_t sensorQueue;

sensorQueue = osMessageQueueNew(
    10,
    sizeof(uint32_t),
    NULL
);

```

Parameters:

```
osMessageQueueNew(  
    message_count,  
    message_size,  
    attributes  
);
```

So:

```
osMessageQueueNew(  
    10,  
    sizeof(uint32_t),  
    NULL  
);
```

means:

```
10 messages  
each message = 4 bytes  
default attributes
```

CMSIS-RTOS2 message queues are FIFO-like and transfer messages from one thread to another.

10. Sending to a Message Queue

The CMSIS function is:

```
osMessageQueuePut()
```

Example:

```
uint32_t temperature = 235;  
  
osMessageQueuePut(  
    sensorQueue,  
    &temperature,  
    0,  
    osWaitForever  
);
```

Parameters:

```
osMessageQueuePut (
    queue_id,
    message_pointer,
    message_priority,
    timeout
);
```

The data is copied into the queue.

For example:

```
temperature
|
| copy
v
+-----+
| Queue |
| 235   |
+-----+
```

The `message_priority` parameter is an interesting difference from the simple FreeRTOS queue API.

CMSIS-RTOS2 can use the message priority to order messages, with higher values having higher priority.

For normal FIFO operation you can simply use:

```
0
```

for the message priority.

11. Receiving from a Message Queue

Use:

```
osMessageQueueGet ()
```

Example:

```
uint32_t temperature;

osMessageQueueGet (
    sensorQueue,
```

```
    &temperature,  
    NULL,  
    osWaitForever  
);
```

Parameters:

```
osMessageQueueGet (  
    queue_id,  
    message_pointer,  
    message_priority_pointer,  
    timeout  
);
```

If no message is available:

```
osWaitForever
```

causes the thread to block.

Conceptually:

```
ControlThread  
  |  
  v  
osMessageQueueGet ()  
  |  
  | no message  
  v  
BLOCKED  
  |  
  | message arrives  
  v  
RUNNING
```

The function returns an `osStatus_t`, so you can check:

```
if (osMessageQueueGet (  
    sensorQueue,  
    &temperature,  
    NULL,  
    osWaitForever) == osOK)  
{  
    ProcessTemperature (temperature);  
}
```

CMSIS-RTOS2 uses status codes such as `osOK`, `osErrorTimeout` and `osErrorResource` rather than FreeRTOS's `pdPASS`.

12. Complete Message Queue Example

Queue

```
osMessageQueueId_t sensorQueue;
```

Create it:

```
sensorQueue = osMessageQueueNew(  
    10,  
    sizeof(uint32_t),  
    NULL  
);
```

Producer

```
void SensorThread(void *argument)  
{  
    uint32_t value;  
  
    while (1)  
    {  
        value = ReadSensor();  
  
        osMessageQueuePut(  
            sensorQueue,  
            &value,  
            0,  
            osWaitForever  
        );  
  
        osDelay(100);  
    }  
}
```

Consumer

```
void ControlThread(void *argument)  
{
```

```

uint32_t value;

while (1)
{
    if (osMessageQueueGet(
        sensorQueue,
        &value,
        NULL,
        osWaitForever) == osOK)
    {
        ProcessSensorValue(value);
    }
}

```

Conceptually:

```

SensorThread
|
| 123
| 124
| 125
v
+-----+
| Queue  |
| 123    |
| 124    |
| 125    |
+-----+
|
v
ControlThread

```

13. Message Queue Timeouts

The timeout parameter works similarly to FreeRTOS.

Don't wait:

```
0
```

Wait forever:

```
osWaitForever
```

Wait a specific amount of time:

```
100
```

Example:

```
osMessageQueueGet (  
    sensorQueue,  
    &value,  
    NULL,  
    100  
);
```

Meaning:

Wait up to 100 kernel ticks for a message.

Important:

The timeout is specified in kernel ticks, not necessarily milliseconds.

If you want a time in milliseconds, use the kernel tick frequency or design your application around the configured tick rate.

14. Message Queue Full

Suppose the queue contains 10 messages:

```
+---+---+---+---+---+  
|  A  |  B  |  C  | ... |  J  |  
+---+---+---+---+---+  
                                FULL
```

Then:

```
osMessageQueuePut (  
    queue,  
    &data,  
    0,  
    100  
);
```

will wait for up to 100 ticks for space.

Alternatively:

```
osMessageQueuePut (
    queue,
    &data,
    0,
    0
);
```

returns immediately.

You can check the result:

```
osStatus_t status;

status = osMessageQueuePut (
    queue,
    &data,
    0,
    0
);

if (status != osOK)
{
    // Queue full / message not accepted
}
```

CMSIS-RTOS2 explicitly defines 0 as try semantics and `osWaitForever` as infinite waiting.

15. Mutexes

What is a mutex?

A mutex protects a shared resource.

Example:

```
Thread A ----+
              |
              v
            UART
              ^
              |
Thread B ----+
```

Without synchronization:

```
Thread A: "Hello"  
Thread B: "Temperature = 23"
```

The output could become corrupted:

```
HeTmeplelorature = 23
```

A mutex ensures that only one thread accesses the resource at a time.

16. Creating a Mutex

CMSIS-RTOS2:

```
osMutexNew()
```

Example:

```
osMutexId_t uartMutex;  
  
uartMutex = osMutexNew(NULL);
```

This replaces:

```
xSemaphoreCreateMutex()
```

from native FreeRTOS.

You can also give the mutex a name:

```
const osMutexAttr_t uartMutex_attributes = {  
    .name = "UART_Mutex"  
};  
  
uartMutex = osMutexNew(  
    &uartMutex_attributes  
);
```

Named objects can be useful when debugging RTOS applications.

17. Taking a Mutex

CMSIS-RTOS2 uses:

```
osMutexAcquire()
```

Example:

```
osMutexAcquire(  
    uartMutex,  
    osWaitForever  
);
```

Then access the UART:

```
HAL_UART_Transmit(  
    &huart1,  
    data,  
    length,  
    100  
);
```

Finally:

```
osMutexRelease(uartMutex);
```

Complete example:

```
void ThreadA(void *argument)  
{  
    while (1)  
    {  
        osMutexAcquire(  
            uartMutex,  
            osWaitForever  
        );  
  
        HAL_UART_Transmit(  
            &huart1,  
            (uint8_t *) "Hello\r\n",
```

```

        7,
        100
    );

    osMutexRelease(uartMutex);

    osDelay(1000);
}
}

```

Another thread:

```

void ThreadB(void *argument)
{
    while (1)
    {
        osMutexAcquire(
            uartMutex,
            osWaitForever
        );

        HAL_UART_Transmit(
            &huart1,
            (uint8_t *) "Temperature\r\n",
            13,
            100
        );

        osMutexRelease(uartMutex);

        osDelay(500);
    }
}

```

18. Mutex vs Message Queue

The distinction remains exactly the same.

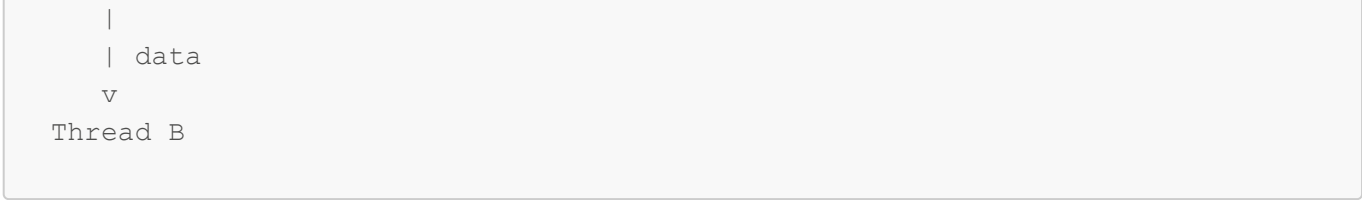
Message Queue

Used to **transfer data**.

```

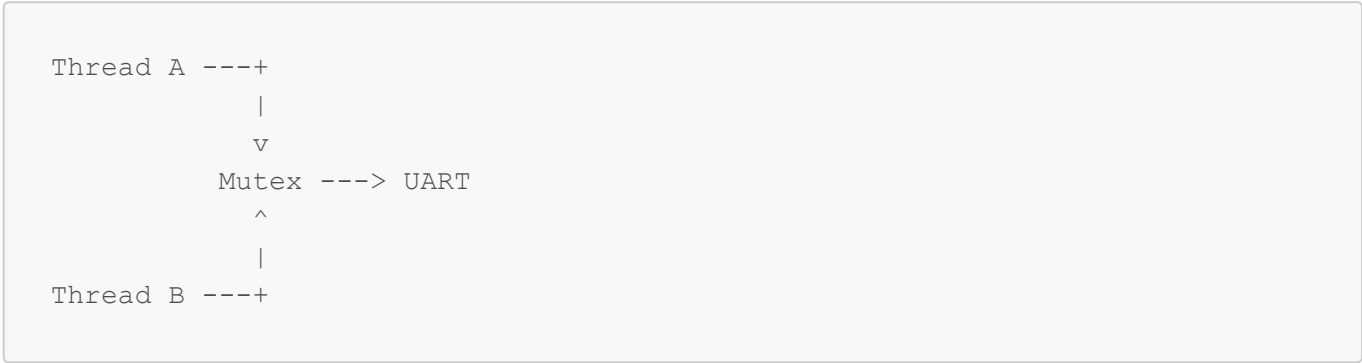
Thread A
|
| data
v
Queue

```



Mutex

Used to **protect a resource**.



Mental model:

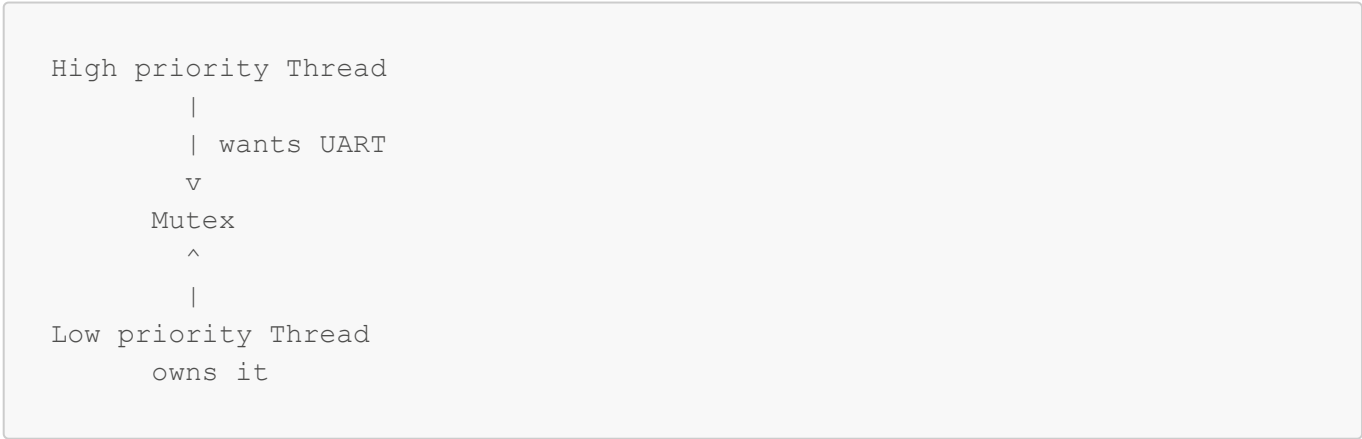
Message Queue = communication

Mutex = ownership/protection

19. Priority Inheritance

A mutex can be used for priority inheritance.

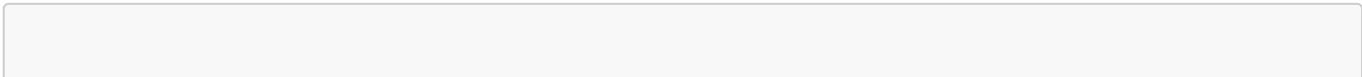
Example:



This can cause **priority inversion**.

CMSIS-RTOS2 mutexes support priority inheritance through the mutex attributes.

For example:



```
const osMutexAttr_t mutex_attributes = {  
    .name = "UART",  
    .attr_bits = osMutexPrioInherit  
};
```

Then:

```
uartMutex = osMutexNew(  
    &mutex_attributes  
);
```

The CMSIS-RTOS2 mutex API supports attributes such as recursive mutexes and priority inheritance.

When using FreeRTOS underneath CMSIS-RTOS2, the CMSIS implementation maps these semantics onto the underlying kernel.

20. Semaphores

CMSIS-RTOS2 provides:

```
osSemaphoreNew()  
osSemaphoreAcquire()  
osSemaphoreRelease()
```

There are two common types:

```
Binary semaphore  
Counting semaphore
```

The purpose is generally synchronization or representing available tokens/resources.

21. Binary Semaphore

A binary semaphore has effectively two states:

```
EMPTY  
|  
| Release  
v  
AVAILABLE
```

```

|
| Acquire
v
EMPTY

```

Typical use:

```

Interrupt
|
| event
v
Semaphore
|
v
Thread

```

The CMSIS API does not have a separate `osBinarySemaphoreNew()` function.

Instead:

```

osSemaphoreNew(
    max_count,
    initial_count,
    attributes
);

```

For a binary semaphore:

```

osSemaphoreId_t adcSemaphore;

adcSemaphore = osSemaphoreNew(
    1,
    0,
    NULL
);

```

Meaning:

```

maximum count = 1
initial count = 0

```

22. Binary Semaphore Example

Create:

```
osSemaphoreId_t adcSemaphore;  
  
adcSemaphore = osSemaphoreNew(  
    1,  
    0,  
    NULL  
);
```

Thread:

```
void AdcThread(void *argument)  
{  
    while (1)  
    {  
        osSemaphoreAcquire(  
            adcSemaphore,  
            osWaitForever  
        );  
  
        ProcessADC();  
    }  
}
```

Some other code signals the thread:

```
osSemaphoreRelease(  
    adcSemaphore  
);
```

The thread wakes up.

Conceptually:

```
ADC event  
|  
v  
Semaphore  
|  
v  
AdcThread  
|  
v  
ProcessADC()
```

23. ISR → Thread Synchronization

This is a particularly useful STM32 pattern.

For example:

```
GPIO interrupt
  |
  | event
  v
Binary Semaphore
  |
  v
ButtonThread
```

CMSIS-RTOS2 simplifies this compared with native FreeRTOS because the API defines which functions are ISR-safe.

For example:

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
    {
        osSemaphoreRelease(
            buttonSemaphore
        );
    }
}
```

Thread:

```
void ButtonThread(void *argument)
{
    while (1)
    {
        osSemaphoreAcquire(
            buttonSemaphore,
            osWaitForever
        );

        ButtonPressed();
    }
}
```

The important difference from native FreeRTOS is that you don't write:

```
xSemaphoreGiveFromISR(...)
```

Instead CMSIS-RTOS2 uses:

```
osSemaphoreRelease(...)
```

The CMSIS-RTOS2 specification explicitly lists `osSemaphoreRelease()` among the APIs that can be called from an ISR.

24. ISR Functions

This is an important difference between the APIs.

With native FreeRTOS:

```
xSemaphoreGive()
```

and:

```
xSemaphoreGiveFromISR()
```

are different functions.

CMSIS-RTOS2 instead defines ISR compatibility as part of the API specification.

For example, these can be used from an ISR:

```
osSemaphoreRelease()  
osMessageQueuePut()  
osThreadFlagsSet()
```

But creation functions such as:

```
osThreadNew()  
osMessageQueueNew()  
osMutexNew()
```

cannot be called from an ISR.

This makes the CMSIS API somewhat cleaner from an application-programming perspective.

25. Counting Semaphores

A counting semaphore can represent multiple available tokens.

Example:

```
osSemaphoreId_t semaphore;  
  
semaphore = osSemaphoreNew(  
    10,  
    0,  
    NULL  
);
```

Meaning:

```
maximum count = 10  
initial count = 0
```

If you release it three times:

```
osSemaphoreRelease(semaphore);  
osSemaphoreRelease(semaphore);  
osSemaphoreRelease(semaphore);
```

the count becomes:

3

Then:

```
osSemaphoreAcquire(semaphore, 0);
```

reduces it to:

2

Useful for:

- counting events
- resource pools
- multiple identical resources
- buffering event occurrences

26. Mutex vs Binary Semaphore

They may look similar:

```
osMutexAcquire()  
osMutexRelease()
```

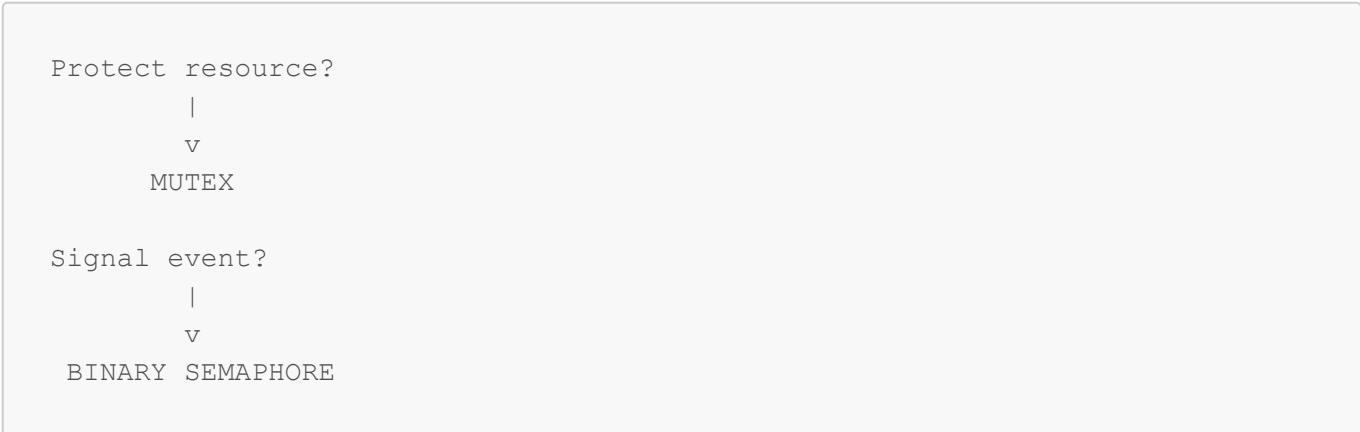
versus:

```
osSemaphoreAcquire()  
osSemaphoreRelease()
```

But they have different semantics.

	Mutex	Binary Semaphore
Purpose	Protect resource	Synchronize/event
Ownership	Yes	No
Priority inheritance	Yes	No
ISR release	No	Yes
Typical use	UART, SPI, shared resource	ISR → thread

Rule:



The distinction is important because a mutex has ownership semantics, while a semaphore is a synchronization/token mechanism.

27. Thread Flags

CMSIS-RTOS2 has an additional synchronization mechanism that is very useful:

Thread Flags

This is something you should definitely know when using CMSIS-RTOS2.

Each thread has its own flag bits.

Conceptually:

```
Thread
+-----+
|  Flags  |
|         |
| bit 0 = RX   |
| bit 1 = TX   |
| bit 2 = ERROR |
| ...         |
+-----+
```

Another thread or ISR can set flags:

```
osThreadFlagsSet(
    threadHandle,
    0x01
);
```

The receiving thread can wait:

```
osThreadFlagsWait(
    0x01,
    osFlagsWaitAny,
    osWaitForever
);
```

This can be a very elegant replacement for multiple binary semaphores.

Example:

```

UART ISR
  |
  +-- set RX flag
  |
  v
CommunicationThread
  |
  +-- RX event

```

Thread flags are part of the CMSIS-RTOS2 API and are specifically intended for thread synchronization.

28. Queues vs Semaphores vs Thread Flags

A useful CMSIS-RTOS2 mental model:

```

Need to transfer DATA?
  |
  v
MESSAGE QUEUE

Need to signal one of several EVENTS?
  |
  v
THREAD FLAGS

Need to signal an EVENT / TOKEN?
  |
  v
SEMAPHORE

Need to protect a RESOURCE?
  |
  v
MUTEX

```

For example:

```

UART received data
  |
  v
Message Queue
  |

```

```

      v
CommunicationThread

```

versus:

```

UART received something
    |
    v
Thread Flag
    |
    v
CommunicationThread

```

The first transfers data.

The second only signals an event.

29. Critical Sections

CMSIS-RTOS2 provides kernel locking mechanisms, but you should generally avoid using them as a replacement for proper synchronization.

For example:

```

osKernelLock();

/* very short critical operation */

osKernelUnlock();

```

However, there is an important conceptual difference compared with:

```

taskENTER_CRITICAL()

```

in native FreeRTOS.

`osKernelLock()` locks the **RTOS scheduler**, while interrupt handling is a separate matter.

For very low-level atomic operations on shared data, use the appropriate Cortex-M/FreeRTOS mechanism rather than blindly protecting large sections of code with kernel locking.

Keep critical sections short.

Don't do:

```
osKernelLock();

HAL_Delay(100);
HAL_UART_Transmit(...);
complexCalculation();

osKernelUnlock();
```

30. Delays vs Synchronization

Avoid polling:

```
while (!dataReady)
{
    osDelay(10);
}
```

Better:

```
osSemaphoreAcquire(
    dataReadySemaphore,
    osWaitForever
);
```

Or:

```
osThreadFlagsWait(
    RX_FLAG,
    osFlagsWaitAny,
    osWaitForever
);
```

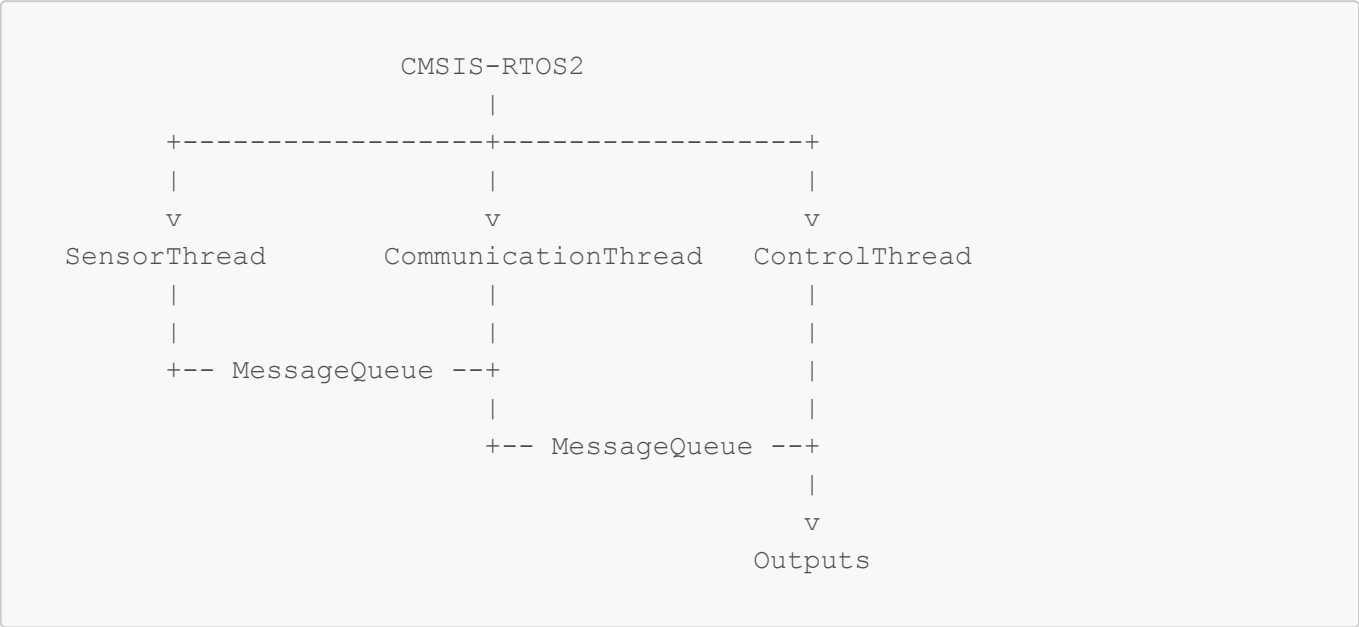
Or, when actual data must be transferred:

```
osMessageQueueGet(
    rxQueue,
    &data,
    NULL,
    osWaitForever
);
```

The thread becomes blocked until the actual event occurs.

31. Typical STM32 Architecture

A real STM32 application might look like:



Hardware interrupts:



32. STM32CubeMX Integration

This is where CMSIS-RTOS2 becomes particularly convenient.

When you enable FreeRTOS in STM32CubeMX, select:

```
CMSIS_V2
```

as the interface/API where applicable.

CubeMX will generate:

```
#include "cmsis_os2.h"
```

and typically a file such as:

```
freertos.c
```

with structures resembling:

```
osThreadId_t defaultTaskHandle;

const osThreadAttr_t defaultTask_attributes = {
    .name = "defaultTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityNormal,
};
```

The thread is then created with:

```
defaultTaskHandle = osThreadNew(
    StartDefaultTask,
    NULL,
    &defaultTask_attributes
);
```

And the thread itself:

```
void StartDefaultTask(void *argument)
{
    for (;;)
    {
        osDelay(1);
    }
}
```

This is the CMSIS-RTOS2 layer.

Underneath it, FreeRTOS is still doing the actual scheduling.

33. Recommended Project Structure

For a larger project, I would separate application functionality from the Cube-generated RTOS setup.

For example:

```
Core/
├── Inc/
│   ├── app_threads.h
│   ├── sensor_thread.h
│   ├── communication_thread.h
│   └── control_thread.h
└── Src/
    ├── app_threads.c
    ├── sensor_thread.c
    ├── communication_thread.c
    └── control_thread.c
```

A thread module might contain:

```
#include "cmsis_os2.h"

static osThreadId_t sensorThreadHandle;

static const osThreadAttr_t sensorThread_attributes = {
    .name = "Sensor",
    .stack_size = 256 * 4,
    .priority = (osPriority_t) osPriorityNormal
};

static void SensorThread(void *argument)
{
    while (1)
    {
        // Sensor processing

        osDelay(100);
    }
}

void SensorThread_Init(void)
{
    sensorThreadHandle = osThreadNew(
        SensorThread,
        NULL,
```

```
        &sensorThread_attributes
    );
}
```

34. A Small Complete Example

Imagine an STM32 application with:

- ADC measurement
- processing thread
- UART output

Architecture:

```
ADC
|
| measurement
v
SensorThread
|
| Message Queue
v
ProcessingThread
|
| Message Queue
v
CommunicationThread
|
v
UART
```

Queue definitions:

```
osMessageQueueId_t adcQueue;
osMessageQueueId_t resultQueue;
```

Initialization:

```
void App_Init(void)
{
    adcQueue = osMessageQueueNew(
        10,
        sizeof(uint16_t),
        NULL
    );
}
```

```
resultQueue = osMessageQueueNew(  
    10,  
    sizeof(uint32_t),  
    NULL  
);  
  
osThreadNew(  
    SensorThread,  
    NULL,  
    &sensorThread_attributes  
);  
  
osThreadNew(  
    ProcessingThread,  
    NULL,  
    &processingThread_attributes  
);  
  
osThreadNew(  
    CommunicationThread,  
    NULL,  
    &communicationThread_attributes  
);  
}
```

Sensor:

```
void SensorThread(void *argument)  
{  
    uint16_t adcValue;  
  
    while (1)  
    {  
        adcValue = ReadADC();  
  
        osMessageQueuePut(  
            adcQueue,  
            &adcValue,  
            0,  
            osWaitForever  
        );  
  
        osDelay(100);  
    }  
}
```

Processing:

```
void ProcessingThread(void *argument)
{
    uint16_t adcValue;
    uint32_t result;

    while (1)
    {
        if (osMessageQueueGet(
            adcQueue,
            &adcValue,
            NULL,
            osWaitForever) == osOK)
        {
            result = ProcessADC(adcValue);

            osMessageQueuePut(
                resultQueue,
                &result,
                0,
                osWaitForever
            );
        }
    }
}
```

Communication:

```
void CommunicationThread(void *argument)
{
    uint32_t result;

    while (1)
    {
        if (osMessageQueueGet(
            resultQueue,
            &result,
            NULL,
            osWaitForever) == osOK)
        {
            SendResultUART(result);
        }
    }
}
```

This is essentially the exact same architecture as the native FreeRTOS example.

Only the API changed.

35. Common CMSIS-RTOS2 API Cheat Sheet

Threads

```
osThreadNew()  
osThreadTerminate()  
osThreadSuspend()  
osThreadResume()  
osThreadYield()  
  
osThreadGetId()  
osThreadGetState()  
osThreadGetPriority()  
osThreadSetPriority()  
osThreadGetName()
```

Delays / timing

```
osDelay()  
osDelayUntil()  
  
osKernelGetTickCount()  
osKernelGetTickFreq()
```

Message Queues

```
osMessageQueueNew()  
  
osMessageQueuePut()  
osMessageQueueGet()  
  
osMessageQueuePeek()  
  
osMessageQueueGetCapacity()  
osMessageQueueGetMsgSize()  
osMessageQueueGetCount()  
osMessageQueueGetSpace()  
  
osMessageQueueReset()  
osMessageQueueDelete()
```

Mutexes

```
osMutexNew()  
  
osMutexAcquire()  
osMutexRelease()  
  
osMutexGetOwner()  
  
osMutexDelete()
```

Semaphores

```
osSemaphoreNew()  
  
osSemaphoreAcquire()  
osSemaphoreRelease()  
  
osSemaphoreGetCount()  
  
osSemaphoreDelete()
```

Thread Flags

```
osThreadFlagsSet()  
osThreadFlagsClear()  
osThreadFlagsGet()  
osThreadFlagsWait()
```

36. The Most Important Design Rules

1. Don't use delays for synchronization

Bad:

```
osDelay(100);  
  
if (dataReady)  
{  
    Process();  
}
```

Better:

```
osSemaphoreAcquire(  
    dataReadySemaphore,  
    osWaitForever  
);  
  
Process();
```

2. Don't poll unnecessarily

Bad:

```
while (!rxComplete)  
{  
    osDelay(1);  
}
```

Better:

```
osThreadFlagsWait(  
    RX_COMPLETE_FLAG,  
    osFlagsWaitAny,  
    osWaitForever  
);
```

or:

```
osSemaphoreAcquire(  
    rxSemaphore,  
    osWaitForever  
);
```

3. Keep ISRs short

Prefer:

```
ISR  
|  
+-- capture information  
+-- signal thread  
|  
+-- return
```

rather than:

```
ISR
|
+-- complex processing
+-- calculations
+-- protocol handling
+-- UART communication
|
+-- return
```

4. Don't hold mutexes longer than necessary

Bad:

```
osMutexAcquire(
    uartMutex,
    osWaitForever
);

DoComplexCalculation();

HAL_Delay(100);

HAL_UART_Transmit(...);

osMutexRelease(uartMutex);
```

Better:

```
DoComplexCalculation();

osMutexAcquire(
    uartMutex,
    osWaitForever
);

HAL_UART_Transmit(...);

osMutexRelease(uartMutex);
```

5. Be careful with stack size

Every thread has its own stack.

CubeMX may generate:

```
.stack_size = 128 * 4
```

for example.

The exact amount you need depends on what the thread calls.

Be particularly careful with:

```
printf()  
large local arrays  
floating-point calculations  
deep function call trees  
recursive functions
```

A useful CMSIS-RTOS2 debugging approach is to use the underlying FreeRTOS facilities if FreeRTOS-specific diagnostics are enabled, or use the RTOS-aware debugger integration in STM32CubeIDE.

6. Avoid starving lower-priority threads

Don't create a high-priority thread like:

```
void HighPriorityThread(void *argument)  
{  
    while (1)  
    {  
        // huge processing loop  
    }  
}
```

A thread that has nothing to do should normally block:

```
osDelay(...);
```

or:

```
osMessageQueueGet(...);
```

or:

```
osSemaphoreAcquire(...);
```

or:

```
osThreadFlagsWait(...);
```

37. A Practical Decision Tree

When writing a new piece of functionality:

```
Does it need its own execution context?  
|  
YES  
|  
THREAD
```

Then:

```
Does Thread A need to send DATA to Thread B?  
|  
YES  
|  
MESSAGE QUEUE
```

```
Does one thread need to wait for an EVENT?  
|  
YES  
|  
SEMAPHORE / THREAD FLAG
```

```
Does multiple code need access to the  
same RESOURCE?  
|  
YES  
|  
MUTEX
```

And:

```

Did the event originate from an ISR?
    |
    YES
    |
Use an ISR-compatible CMSIS-RTOS2 API

```

For example:

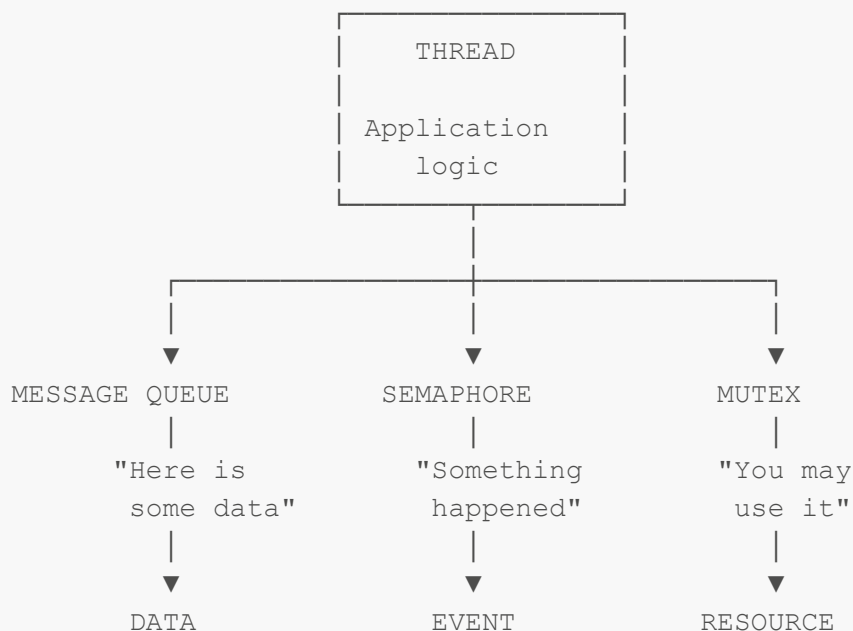
```

ADC interrupt
    |
+----- data ---> Message Queue
    |
+----- event --> Thread Flag
    |
+----- event --> Semaphore

```

38. One Mental Model to Remember

If you remember only this:



And with CMSIS-RTOS2:

```

DATA      → osMessageQueue
EVENT     → osSemaphore / osThreadFlags
RESOURCE  → osMutex
EXECUTION → osThread

```

That is the core of most CMSIS-RTOS2 applications.

39. Recommended First STM32 CMSIS-RTOS2 Experiment

Build exactly the same experiment as with native FreeRTOS, but use CMSIS-RTOS2.

Step 1 — LED Thread

```
LED Thread
|
+-- toggles LED every 500 ms
```

Using:

```
osDelay(500);
```

Step 2 — Producer Thread

```
Producer Thread
|
+-- generates counter
|
+-- osMessageQueuePut()
```

Step 3 — Consumer Thread

```
Consumer Thread
|
+-- osMessageQueueGet()
|
+-- prints counter over UART
```

Step 4 — Button Interrupt

Step 5 — UART Mutex

```

      UART
      ^
      |
    Mutex
      ^
+-----+-----+
|               |
Thread A      Thread B

```

```
osMutexAcquire()  
osMutexRelease()
```

Native FreeRTOS → CMSIS-RTOS2 Translation Table

Concept	Native FreeRTOS	CMSIS-RTOS2
Task	<code>TaskHandle_t</code>	<code>osThreadId_t</code>
Create task	<code>xTaskCreate()</code>	<code>osThreadNew()</code>
Delete task	<code>vTaskDelete()</code>	<code>osThreadTerminate()</code>
Delay	<code>vTaskDelay()</code>	<code>osDelay()</code>
Periodic delay	<code>vTaskDelayUntil()</code>	<code>osDelayUntil()</code>
Task yield	<code>taskYIELD()</code>	<code>osThreadYield()</code>

Concept	Native FreeRTOS	CMSIS-RTOS2
Queue handle	<code>QueueHandle_t</code>	<code>osMessageQueueId_t</code>
Create queue	<code>xQueueCreate()</code>	<code>osMessageQueueNew()</code>
Send queue	<code>xQueueSend()</code>	<code>osMessageQueuePut()</code>
Receive queue	<code>xQueueReceive()</code>	<code>osMessageQueueGet()</code>
Queue peek	<code>xQueuePeek()</code>	<code>osMessageQueuePeek()</code>
Mutex handle	<code>SemaphoreHandle_t</code>	<code>osMutexId_t</code>
Create mutex	<code>xSemaphoreCreateMutex()</code>	<code>osMutexNew()</code>
Take mutex	<code>xSemaphoreTake()</code>	<code>osMutexAcquire()</code>
Give mutex	<code>xSemaphoreGive()</code>	<code>osMutexRelease()</code>
Semaphore handle	<code>SemaphoreHandle_t</code>	<code>osSemaphoreId_t</code>
Create semaphore	<code>xSemaphoreCreateBinary()</code>	<code>osSemaphoreNew(1, 0, ...)</code>
Take semaphore	<code>xSemaphoreTake()</code>	<code>osSemaphoreAcquire()</code>
Give semaphore	<code>xSemaphoreGive()</code>	<code>osSemaphoreRelease()</code>
Task notification	<code>xTaskNotify()</code>	<code>osThreadFlagsSet()</code> / other CMSIS mechanisms
Wait notification	<code>ulTaskNotifyTake()</code>	<code>osThreadFlagsWait()</code>
Infinite wait	<code>portMAX_DELAY</code>	<code>osWaitForever</code>
Success	<code>pdPASS</code>	<code>osOK</code>
Headers	<code>FreeRTOS.h</code> , <code>task.h</code> , etc.	<code>cmsis_os2.h</code>

One Very Important Difference

The native FreeRTOS API gives you direct access to FreeRTOS concepts:

```
xTaskCreate()  
xQueueCreate()  
xSemaphoreTake()
```

CMSIS-RTOS2 gives you a **portable RTOS abstraction**:

```
osThreadNew()  
osMessageQueueNew()  
osSemaphoreAcquire()
```

That abstraction is intentional. CMSIS-RTOS2 is designed as a generic API layer that can interface with an RTOS kernel rather than exposing the kernel-specific API directly.

Therefore, if you write:

```
osMessageQueuePut(...)
```

your application doesn't need to care whether the CMSIS-RTOS2 implementation underneath is FreeRTOS, RTX, or another compliant RTOS.

For your STM32/CubeMX project, however, the practical stack is usually:

```
Your Application  
    |  
    v  
CMSIS-RTOS2  
    |  
    v  
FreeRTOS  
    |  
    v  
STM32 Cortex-M
```

And that is why the two cheat sheets feel so similar: **they are controlling essentially the same RTOS machinery through two different APIs.**